RMIT UNIVERSITY

ISYS3476 (Postgraduate)
Managing Semi-structured and Unstructured Data
Assignment 1 Part A: Pre-processing & Feature Engineering
Rubric

1 What this rubric contains

This rubric provides the detailed marking criteria for Part A. The required implementations and equations are stated in the Main Spec.

2 Marking Rubrics

2.1 At-a-glance marking summary

Component	Points
Task 1 preprocessing (retrieval-based performance 4; procedure docstring 2)	6
Task 2 correctness	4
Task 3 correctness	3
Task 4 correctness	3
Engineering development evidence	2
Code quality	2
Total	**20**

2.2 Task 1: Preprocessing rubric (6 points)

Task 1 retrieval-based performance combines two disclosed noise-profile categories. Hidden content and noise placements differ from public dev. Each query is evaluated once per profile, and the combined metric averages all profile observations. The `preprocess` procedure docstring is inspected manually. Average Hit@K follows the Week 02 workshop convention.

Retrieval-based performance (4 points)

The criteria below assume `preprocess` runs on the hidden retrieval test corpus without crashing and within 3 minutes.

Points	Criteria
4	Combined Average Hit@K ≥ 0.55.
3	Combined Average Hit@K ≥ 0.40.
2	Combined Average Hit@K ≥ 0.25.
1	Runs, but combined Average Hit@K < 0.25.
0	Crashes, times out, cannot be imported, or cannot be evaluated.

Procedure docstring (2 points)

Points	Criteria
2	The docstring gives a code-consistent explanation grounded in concrete observations from both public profiles, justifies the main operation order, and explains why the choices should generalise beyond individual dev examples.
1	The docstring gives a broadly relevant procedure, but its ordering rationale, cross-profile evidence, generalisation, or consistency with the submitted implementation is incomplete.
0	The docstring is missing, generic, or substantially inconsistent with the submitted preprocessing code.

2.3 Task 2: Character N-grams and Positions (4 points)

We assess correctness of: `make_ngrams_chars` and `make_positions`. Each function is worth 2 points.

Per-function pass-rate	Points	Notes
`pass_rate` $= 1.00$	2	All test cases correct (including edge cases).
$0.40 \leq$ `pass_rate` < 1.00	1	Partially correct; common issues include incorrect boundary markers, wrong ordering, or wrong position indexing.
`pass_rate` < 0.40 or not runnable	0	Insufficient correctness or crashes.

2.4 Task 3: TF–IDF variants (3 points)

We assess only: `tf_mode` $\in \{$`'log'`, `'bm25'`$\}$.

Mode	Pass-rate	Points	Criteria
`bm25`	`pass_rate` = 1.00	2	All BM25 tests correct.
	0.50 ≤ `pass_rate` < 1.00	1	Partially correct BM25 behaviour.
	`pass_rate` < 0.50 or crash	0	Insufficient BM25 correctness.
`log`	`pass_rate` = 1.00	1	All log-TF tests correct.
	`pass_rate` < 1.00 or crash	0	Any incorrectness in log-TF mode receives 0 for this 1-point component.

2.5 Task 4: Semantic vectors (3 points)

We assess only: `method` ∈ {`'tfidf_weighted'`, `'meanmax'`}.
Test cases include output-shape checks, OOV/`<unk>` handling, and pooling/weighting behaviour.

Method	Pass-rate	Points	Criteria
`tfidf_weighted`	`pass_rate` = 1.00	2	All tests correct (includes OOV/`<unk>`, shape, and weighting behaviour).
	0.50 ≤ `pass_rate` < 1.00	1	Partially correct behaviour.
	`pass_rate` < 0.50 or crash	0	Insufficient correctness.
`meanmax`	`pass_rate` = 1.00	1	All tests correct (shape and pooling).
	`pass_rate` < 1.00 or crash	0	Any incorrectness receives 0 for this 1-point component.

2.6 Engineering development evidence (2 points)

This item is assessed from the short engineering development note in `DEVELOPMENT.md`. It concerns implementation and debugging evidence, not method design or experimental evaluation.

Points	Criteria
2	Provides both required excerpts. Each includes concrete failed output, a brief implementation change, and the rerun result.
1	Provides one complete excerpt, or both excerpts with one of these elements missing.
0	Missing, unrelated, contains no concrete failed output, or describes only method design, experimental evaluation, or final successful output.

2.7 Code quality across all tasks (2 points, holistic)

This is a manual, holistic assessment across your Part A implementation.

Points	Criteria
2	Clear, consistent, and maintainable code. Includes useful brief documentation, sensible handling of noisy or invalid input, and no obvious dead code or unused imports. Preserves the required folder structure and provided agent instruction and configuration files.
1	Runnable and readable code with basic documentation. Preserves the required folder structure and provided agent instruction and configuration files, but has minor style, maintainability, or robustness issues.
0	Difficult to follow, poorly documented, or frequently crashes. Uses disallowed libraries, introduces inappropriate side effects, breaks the required folder structure, or alters provided agent instruction or configuration files.